C2 Advanced Research Computing

Word Count: 2482 (including figure captions)

Funmi Looi-Somoye

1st April 2026

Contents

1 Introduction **3**

2 Cholesky Background and Baseline Implementation **3**
2.1 Cholesky Background . . . 3
2.2 Baseline Implementation . . . 3

3 Serial Optimisation **4**
3.1 Serial Optimisation Implementation . . . 4
3.2 Header and Benchmarking Files . . . 5
3.3 Serial Performance Analysis . . . 5

4 OpenMP Optimisation **6**
4.1 OpenMP Implementation . . . 6
4.2 Header and Benchmarking Files . . . 7
4.3 OpenMP Performance Analysis . . . 7

5 Performance Results **9**
5.1 Serial . . . 9
5.2 Block Size . . . 10
5.3 OpenMP Parallelisation . . . 10

6 Conclusion **12**

7 Development Practices **12**
7.1 Testing . . . 12
7.2 Code Formatting . . . 12
7.3 CMake . . . 13

8 HPC Experimental Setup **13**
8.1 Hardware . . . 13

8.2 Software 13
8.3 Execution Settings 13
8.4 Scripts 13

9 Git Tags and Versioning **14**

10 Acknowledgments **14**

1 Introduction

This report presents the work completed for the C2 Advanced Research Computing module. The focus of the report is the implementation and optimisation of Cholesky decomposition, starting from a baseline version and progressing through serial and OpenMP-based optimisations. At each stage the development process is described, results are presented, and the performance is analysed.

2 Cholesky Background and Baseline Implementation

2.1 Cholesky Background

Covariance matrices arise often in scientific data analysis. For instance, when comparing model predictions against observed data. Calculating statistical likelihoods often requires computing the log-determinant of these matrices, which can be done efficiently using the Cholesky factorisation.

Cholesky factorisation exploits the symmetric positive-definite structure of a matrix, reducing the computational cost when compared to a general LU decomposition - in general it can halve the number of required floating-point operations.

The key idea for Cholesky factorisation is that a covariance matrix C can be decomposed into the product of a lower triangular matrix L and its transpose,

$$C = LL^T. \tag{1}$$

This decomposition provides an efficient way to compute the determinant, since the determinant of a triangular matrix is the product of its diagonal elements. As a result, the log-determinant of C can be written as

$$\log |C| = 2 \sum_{p=0}^{n-1} \log L_{pp}. \tag{2}$$

It is important to note that Cholesky factorisation is only defined for matrices that are symmetric ($C = C^T$), square, and positive definite.

2.2 Baseline Implementation

The baseline implementation follows the structure outlined in the coursework brief. This implementation is single-threaded and was used to provide a clear baseline to compare other optimisations.

As outlined in the brief, the Cholesky implementation operates in-place on a row-major $n \times n$ matrix, updating one pivot index p at a time. For each p, it computes the diagonal term $L_{pp} = \sqrt{c_{pp}}$, scales the corresponding row and column entries, and then performs an update on

the trailing submatrix. This produces the lower-triangular Cholesky factor while preserving the in-place storage structure.

In the baseline version and all further optimisations, a matrix size limit was included to ensure users do not input matrices greater than $n = 100,000$. Timing is measured around the factorisation kernel and returned in seconds, which is later used for direct optimisation comparisons. The expected computational complexity for this implementation is $O(n^3)$.

For code reference, the baseline implementation is provided in *include/cholesky/V1_baseline.hpp*, with the corresponding benchmarking file in *src/matrix/V1_baseline.cpp*.

3 Serial Optimisation

This section describes the improvements made to the baseline implementation and evaluates their effect on performance.

3.1 Serial Optimisation Implementation

The objective of these optimisations was to keep the same in-place Cholesky algorithm while reducing runtime through better memory access patterns, fewer repeated operations, and clearer data flow.

The optimisation process was incremental. Versions 2 to 6 applied each change on top of the baseline implementation, and version 7 is a combination of all optimisation techniques explored.

- **V2 (loop order):** The loop order was changed in the trailing-submatrix update so that memory is accessed in a more cache-friendly row-major pattern.
- **V3 (scalar hoisting):** Applied hoisted repeated loads (for example, storing c_{pj} once per outer-loop iteration) to reduce redundant memory reads inside the innermost loop.
- **V4 (pointer-based indexing):** Introduced row pointers to reduce repeated index arithmetic.
- **V5 (merged scaling loop):** Merged the row and column scaling loops after diagonal normalisation, reducing loop overhead in the panel update stage.
- **V6 (cache blocking):** Added tiled updates (i/j blocking) in the trailing-submatrix computation to improve cache reuse on larger matrices.
- **V7 (combined serial version):** Applied a combination of optimisations from blocking, pointer access, and scalar hoisting into a blocked serial kernel, panel solve, and blocked trailing update.

It is important to note that all serial versions were benchmarked under consistent conditions. In particular, builds used the same compiler and optimisation flags and experiments were run with a single thread/core, (OMP_NUM_THREADS=1 –cpus-per-task=1), refer to the submission scripts for further details.

To keep performance comparisons consistent, each version uses the same matrix generator and the same matrix-size sweep (128 to 4096). To measure runtime, three timing repeats were recorded per size and an average runtime found.

For blocked versions, a dedicated block-size sweep was run to select an ideal block size before moving to OpenMP parallelisation.

Therefore reported timings are directly comparable across versions. Even though these optimisations did not change the $O(n^3)$ complexity of Cholesky factorisation, they reduced runtime by improving data locality and reducing instruction overhead.

This approach made it easier to isolate which optimisation produced meaningful gains and to confirm that correctness was preserved at each step.

3.2 Header and Benchmarking Files

The serial optimisation kernels are implemented in the include/cholesky/ folder.

The corresponding matrix-size benchmarking files are in the src/matrix/ folder.

Block-size sweep experiments are provided in the src/blocking/ folder.

3.3 Serial Performance Analysis

Figures 1 and 2 show average runtime (three runs per matrix size) and speedup relative to the baseline ($S = T_{\mathrm{V1}}/T_{\mathrm{version}}$). Overall, serial optimisations produced a clear and consistent reduction in runtime at larger matrix sizes, with the combined serial kernel (V7) giving the best performance.

The most prominent result is the gap between V1 and V7 at the largest matrices. At $n = 4096$, runtime decreases from 419.63 s (V1) to 10.63 s (V7), corresponding to a 39.48× speedup. At $n = 2048$, runtime drops from 16.15 s to 0.903 s (17.89×), and at $n = 1024$, from 1.91 s to 0.113 s (16.97×). This indicates that a combined optimisation strategy is beneficial, particularly when matrix size grows (Figure 1).

Analysing the other versions, V2 (loop-order change) results in a large gain, reaching 28.46× speedup at $n = 4096$, which suggests memory-access order is a dominant factor for this Cholesky implementation. In contrast, V3 (scalar hoisting alone) is slightly slower than baseline (0.71× to 0.99× relative speed). Similarly, the V4 pointer change produced only marginal performance differences relative to V1. At $n = 512$, V4 is slightly faster 0.195 s vs 0.199 s, 1.02×, and at $n = 4096$ the runtime is effectively unchanged 417.81 s vs 419.63 s. The V5 (merged loop) hardly improves runtime with a small decrease in runtime at larger sizes (for example a reduction of 0.97× at $n = 2048$). V6 (cache blocking) produces some gains, with a 1.96× speed up at $n = 4096$. In summary, V7 combines blocking, pointer-based access, and scalar reuse, and delivers the best runtime reduction across all matrix sizes (Figures 1 and 2).

The observed trends are mainly explained by memory behaviour rather than changes in complexity. In V1, the trailing-submatrix update accesses memory with poor locality, so larger matrices suffer

from frequent cache misses. V2 improves loop ordering and therefore improves spatial locality, which explains the large speedup observed. Conversely, V3 (scalar hoisting), V4 (pointers) and V5 (merged scaling loop) do not significantly improve memory behaviour, so gains are limited and can even be negative due to added overhead.

V6 introduces cache blocking, which improves data reuse in the trailing update. This becomes increasingly effective as n grows. V7 combines blocking with pointer indexing and scalar reuse, so it reduces both memory stalls and repeated operations. This explains why V7 gives the best performance at all large matrix sizes.

For smaller matrices, runtimes are already very small, so optimisation overhead and timer noise have a proportionally larger effect. At $n = 128$, for instance, V6 is actually slower than V1 (1.091 ms versus 1.023 ms), whereas by $n = 256$ V6 performs better (10.730 ms versus 14.015 ms), suggesting that blocking begins to become beneficial around $n > 256$. It is worth noting that V7 remains the fastest even at small sizes, though in practical applications the impact would be unnoticeable at such scales. Therefore these results suggest that the main benefit of serial optimisations is at medium to large matrix sizes.

Analysing the results from the block-size sweep, we can see the performance is sensitive to block size, but the trend differs between versions. For V6, the sweep gives the fastest runtime at $B = 8$ (0.691 s). Moving to $B = 16$ and $B = 32$, runtime jumps to about 1.29 s, then improves slightly around $B = 64$ to $B = 128$ (1.230 s to 1.220 s), but still does not match $B = 8$. Performance then drops off again for larger blocks. For V7, the pattern is cleaner. Runtime improves steadily from $B = 8$ (0.187 s) to an optimum at $B = 128$ (0.108 s), then worsens again at $B = 256$ and $B = 512$ (Figure 3).

Overall, this suggests that moderate block sizes provide the best cache reuse without introducing excessive loop overhead.

4 OpenMP Optimisation

This section describes the OpenMP parallelisation of the Cholesky implementation and evaluates the resulting performance.

4.1 OpenMP Implementation

After finding the best performing serial optimisation (V7), OpenMP was introduced incrementally from V8 to V11. The aim was to parallelise only those regions that are independent at block level, while preserving correctness in the diagonal block factorisation.

The OpenMP optimisations were:

- **V8 (parallel naive):** Starting from the V7 serial implementation, OpenMP parallel loops were added over independent block rows in the panel solve and trailing-submatrix update.
- **V9 (parallel static):** The same parallel structure as V8 was maintained, but the schedule(static) was added to reduce scheduling overhead and enforce predictable work

partitioning.

- **V10 (parallel dynamic):** Similar to V9, V10 keeps the same parallel structure as V8, but uses schedule(dynamic) to test whether dynamic load balancing improves performance when block work is uneven.
- **V11 (parallel combined):** A combined strategy was implemented, schedule(static) for the panel solve, and collapse(2) and schedule(dynamic) for the trailing update across (i, j) block pairs. This increased available parallel tasks in the most computationally expensive part.

Across all OpenMP versions, the algorithmic structure remains the same V7 Cholesky implementation with the same in-place matrix layout and $O(n^3)$ complexity. The key changes are therefore in thread-level work distribution and scheduling policy.

For fair comparison, matrix-size experiments use the same sizes as earlier sections (128 to 4096), with three repeats per size, fixed block size $B = 64$, and a fixed OpenMP thread setting (NUM=16) in the matrix-size test. In practice, B=64 gave consistently strong performance and good thread-scaling behaviour, so it was chosen as the fixed experimental setting.

Separate thread-scaling experiments sweep NUM=1 to 70 at fixed $n = 4096$, again with $B = 64$, to evaluate scalability and scheduling behaviour. In these single-node runs, OMP_NUM_THREADS was matched to the requested CPU cores (–cpus-per-task).

4.2 Header and Benchmarking Files

The OpenMP kernels are implemented in the include/cholesky/ folder.

The corresponding matrix-size benchmarking files are in the src/matrix/ folder

Thread-scaling experiments are provided in the src/threads folder.

4.3 OpenMP Performance Analysis

For the matrix-size runs, all OpenMP versions (V8–V11) were substantially faster than both the V1 baseline and V7 serial at medium and large n (Figure 4). At $n = 4096$, runtimes were close across versions. V8 achieved 0.935 s, V9 0.937 s, V11 0.831 s, and V10 was the fastest at 0.825 s. At $n = 2048$, V11 led with 0.063 s, ahead of V10 (0.073 s) and both V8 and V9 at 0.080 s.

The resulting speedups were substantial. Relative to V1 at $n = 4096$, V8 and V9 achieved approximately 449× and 448× respectively, whilst V10 and V11 reached 509× and 505×. Relative to V7 at the same size, these figures were 11.38×, 11.34×, 12.90×, and 12.80× respectively. V11 also showed strong gains at medium sizes, reaching 14.33× over V7 at $n = 2048$.

From the thread-scaling runs at $n = 4096$ and 16 threads, V10 performed best with a speedup of 14.11× and efficiency $E = 0.882$, followed by V8 (13.57×, $E = 0.848$), V9 (13.56×, $E = 0.847$), and V11 (12.91×, $E = 0.807$). The best observed runtime across the full 1–70 thread range was achieved by V11 at 69 threads, yielding 0.325 s, a speedup of 34.30×, and an efficiency of $E = 0.497$ (Figures 5, 6, and 7).

These trends were consistent with the scheduling choices in each version. V8 and V9 parallelised the same block-row loops, so their results remained very similar. Schedule(static) in V9 mainly reduced scheduling overhead but did not fundamentally change available parallel work.

V10 using schedule(dynamic) performed best at moderate thread counts (notably 16 threads) because dynamic scheduling improved load balancing when block-level work was uneven. This explained its higher efficiency at 16 threads (0.882).

V11 combined static panel solve with dynamic collapsed trailing update, which increased task granularity in the most intensive step. This was particularly beneficial at higher thread counts, where V11 achieved the best absolute runtime and strongest thread speedup, even though efficiency still dropped as the number of threads increased.

For smaller matrices, all runtimes were already very low, so parallel overhead and measurement noise were more noticeable, and the differences between scheduling policies were less meaningful.

Including ideal reference lines in Figures 6 and 7, the measured speedup curves remained below the ideal linear line, with efficiency falling below 1.0 as thread count increased. This was expected as OpenMP overheads, synchronisation costs, and remaining serial work prevent perfect scaling, and Amdahl's Law predicts that speedup will plateau as thread count rises due to the remaining serial portion of the code. In practice, the best-performing versions tracked the ideal trend reasonably well at low to moderate thread counts before diverging more clearly at higher thread counts, consistent with this prediction. Since V11 achieved the best speedup at 69 threads, it suggests that combined scheduling was effective at maximising parallel work and minimising overhead.

5 Performance Results

5.1 Serial

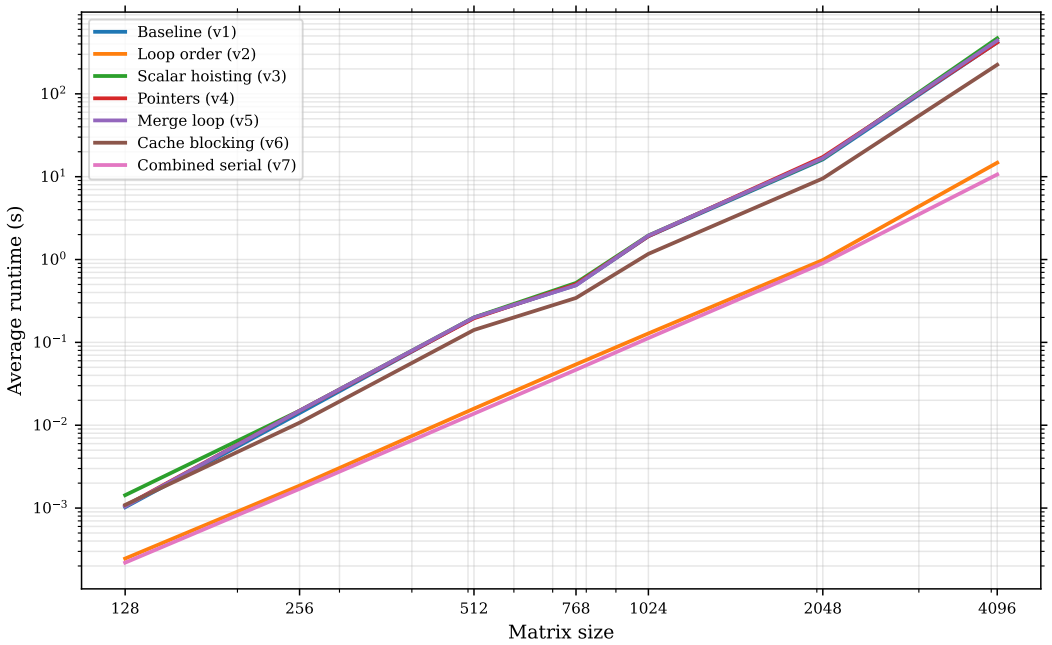

Figure 1: Runtime comparison across matrix sizes (128, 256, 512, 768, 1024, 2048, 4096) for baseline and serially optimised versions.

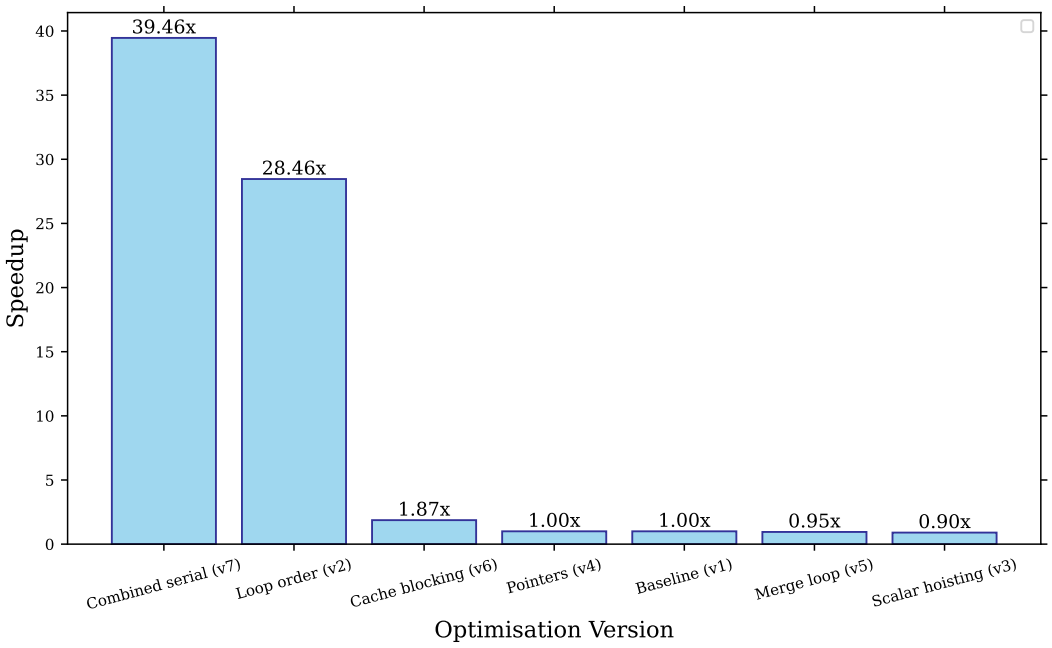

Figure 2: Speedup at the largest matrix size $n = 4096$ for serial versions.

5.2 Block Size

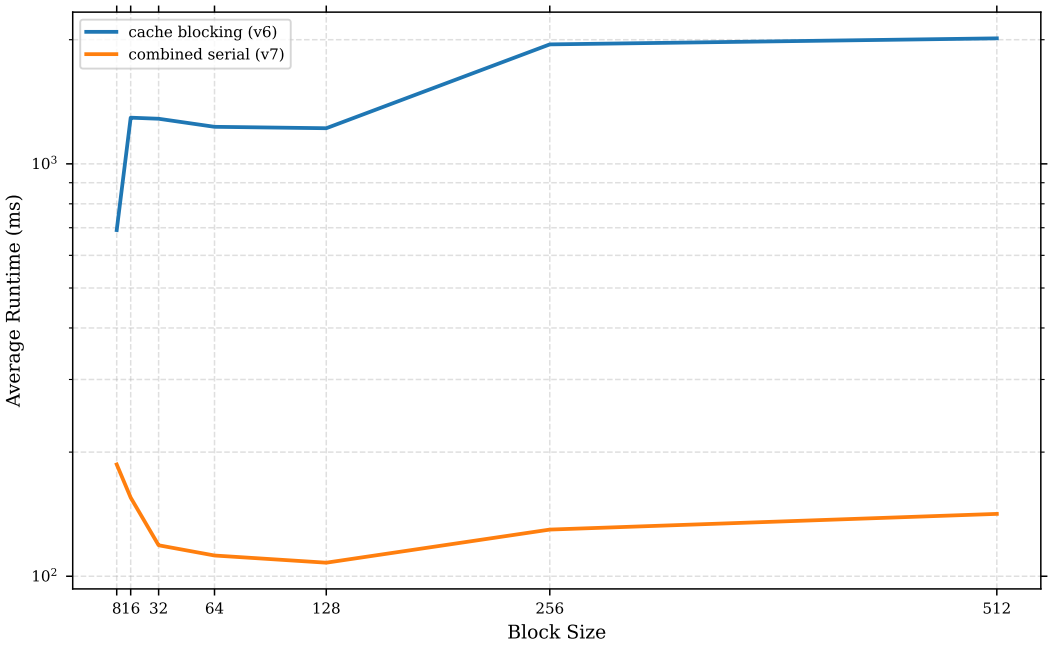

Figure 3: Effect of block size (8, 16, 32, 64, 128, 256, 512) on cache-blocked serial optimisations for matrix $n = 1024$.

5.3 OpenMP Parallelisation

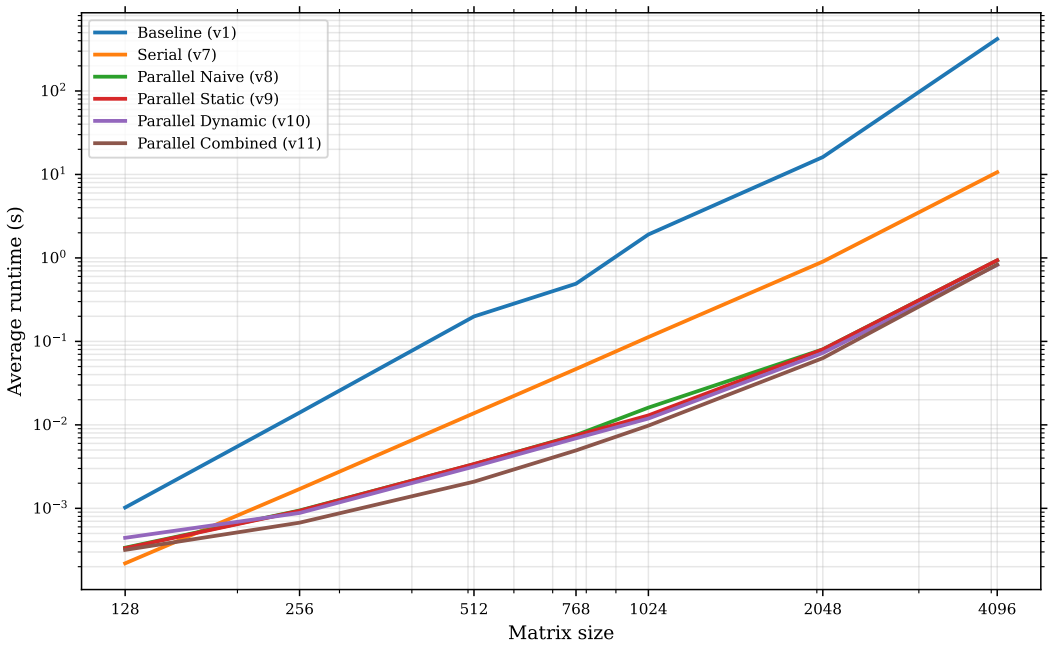

Figure 4: OpenMP runtime comparison across matrix sizes (128, 256, 512, 768, 1024, 2048, 4096). For block size = 64 and 16 threads. Baseline and best serial version included for reference.

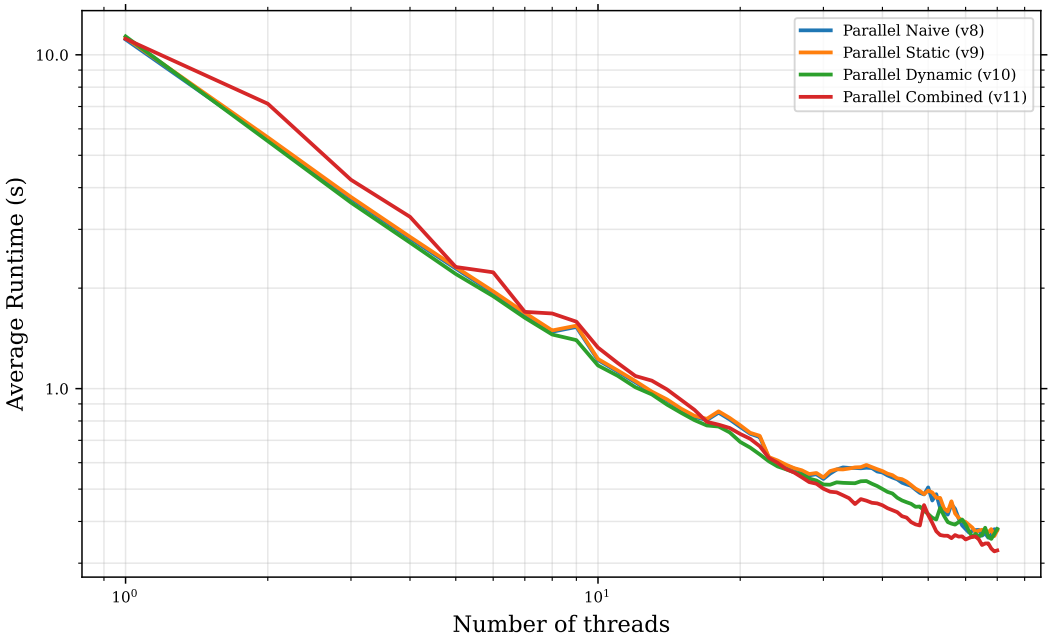

Figure 5: OpenMP runtime for different thread counts (1 to 70) for parallel versions for the largest matrix size $n = 4096$ and block size $= 64$.

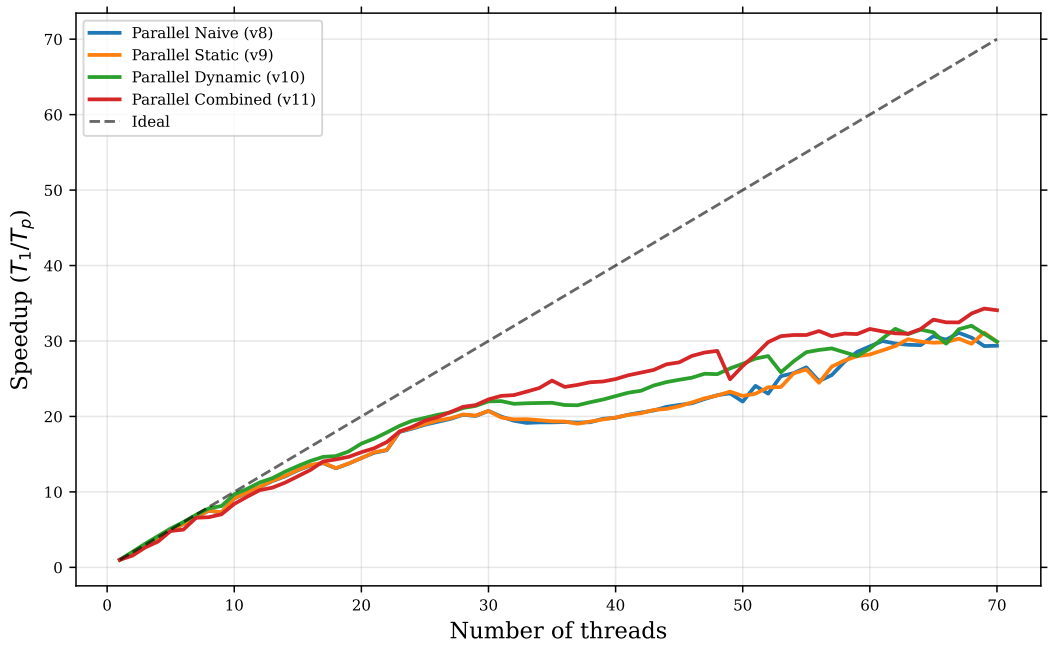

Figure 6: Thread-scaling speedup comparison across OpenMP versions for the largest matrix size $n = 4096$ and block size $= 64$.

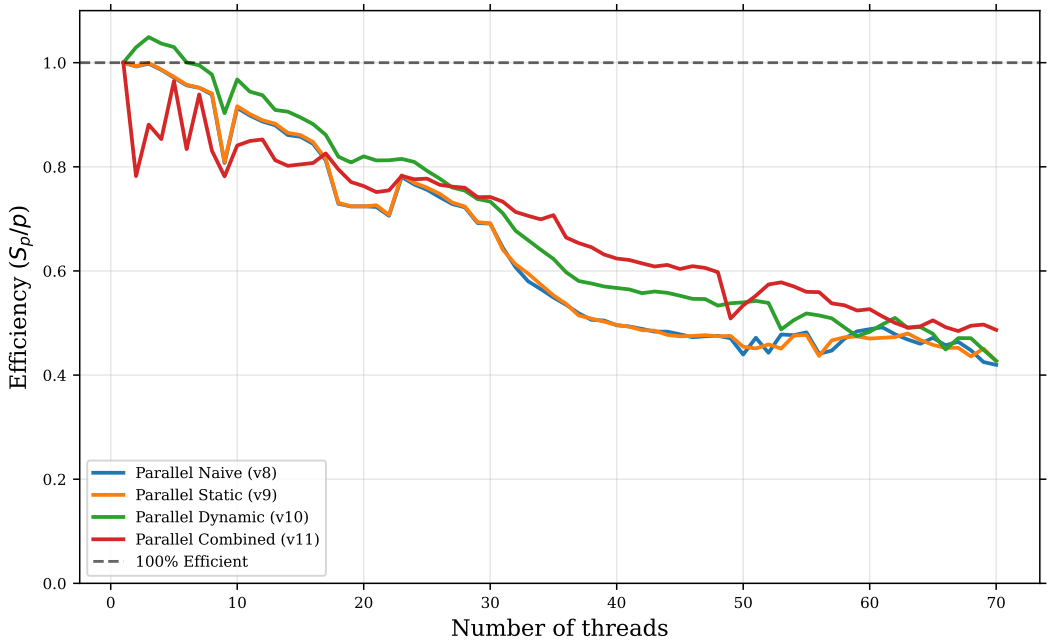

Figure 7: Parallel efficiency for different thread count across OpenMP versions for the largest matrix size $n = 4096$ and block size $= 64$.

6 Conclusion

This project showed that effective serial optimisation can result in large gains over the baseline, especially at larger matrix sizes. The best serial version (V7) substantially reduced runtime, and OpenMP parallelisation (V8–V11) then provided further improvement, with the best parallel versions giving strong speedup at high thread counts. Overall, the results show that memory blocking with appropriate OpenMP scheduling are the key factors for performance in this Cholesky implementation.

7 Development Practices

7.1 Testing

The test scripts are located in the test/ directory and use Google Test to check that each Cholesky implementation is producing numerically correct results. The tests build symmetric positive-definite matrices, run each version of the code, and then compute the log-determinant from the returned lower-triangular factor to compare against a LAPACK reference (dpotrf) within a small tolerance.

7.2 Code Formatting

To ensure the project follows software engineering best practice, clang-format has been implemented to automatically manage code layout and indentation. This ensures the code remains clean and readable. Additionally, Doxygen-style commenting has been adopted, which allows structured documentation to be generated directly from source code.

7.3 CMake

CMake was used to manage the build system consistently across the baseline, serial, and OpenMP implementations, which made it straightforward to compile and run different benchmark targets. It also allowed compiler options and target-specific settings to be organised in one place, so experiments could be reproduced reliably throughout the project.

8 HPC Experimental Setup

This section records the hardware, software, and execution environment used for the experiments.

8.1 Hardware

- System: CSD3 cluster, icelake partition, single-node.
- CPU: Intel Xeon Ice Lake class CPU nodes.
- Number of cores: Up to 70 CPU cores per task were allocated (–cpus-per-task=70) for thread-scaling runs, 16 for matrix-size OpenMP runs and 1 for serial/block-sweep runs.

8.2 Software

- Compiler flags: -O3 used in benchmark submission scripts.

8.3 Execution Settings

- Thread counts: Matrix-size OpenMP runs used 16 threads, thread-scaling runs swept 1–70 threads and block-sweep runs used 1 thread.
- Core usage: OMP_NUM_THREADS was set to match –cpus-per-task, so thread count and cores used are equivalent in the reported runs.
- Matrix sizes tested: 128, 256, 512, 768, 1024, 2048, 4096 for matrix-size experiments, 4096 for thread sweeps, 1024 for block-size sweeps.
- Job submission settings: Slurm account MPHIL-DIS-SL2-CPU, partition icelake, –nodes=1, –ntasks=1, –no-requeue; walltimes of 3h (matrix), 2h (threads), and 10m (blocking).

8.4 Scripts

All shell submission scripts are stored in the scripts/ directory of the project:

- scripts/runtime_matrix.sh: Builds the project and runs all executables in build/matrix/src/matrix.
- scripts/runtime_threads.sh: Builds the project and runs all executables in build/threads/src/threads.
- scripts/runtime_block.sh: Builds and runs the block-size benchmarking executables in build/blocking/src/blocking.

9 Git Tags and Versioning

The following Git tags were used during development:

Table 1: Git tags used in this project and their purpose.

Tag	Description
V1	Baseline in-place Cholesky implementation.
V2	Baseline implementation with loop reordering for better memory locality.
V3	Baseline implementation with scalar hoisting to reduce repeated loads.
V4	Baseline implementation with pointer indexing to reduce index arithmetic.
V5	Baseline implementation with merged row and column scaling loops.
V6	Baseline implementation with cache blocking in the trailing submatrix update.
V7	Combined serial optimisations with blocking, pointer access, and scalar reuse.
V8	OpenMP parallel version of V7 over panel solve and trailing update block loops.
V9	V8 with OpenMP schedule(static) for predictable work partitioning.
V10	V8 with OpenMP schedule(dynamic) for improved load balancing.
V11	Combined OpenMP version using static panel solve and dynamic collapsed trailing update.

10 Acknowledgments

Timing for this project used files sourced from the MPhil Data Intensive Science, C2 Advanced Research Computing GitLab page.